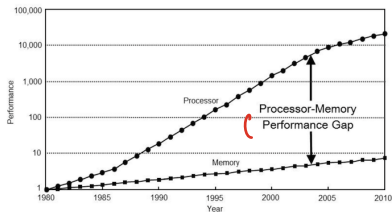


Memory

2025 Semester 1 SOFTENG 370: Operating Systems
Talia Xu

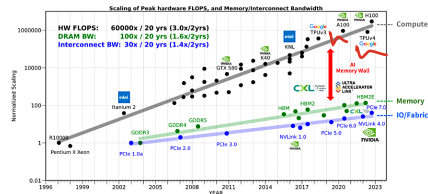
Lecture 10
2.0.0

The Memory Wall Problem



Computer Architecture: A Quantitative Approach by John L. Hennessy, David A. Patterson, Andrea C. Arpaci-Dusseau

The Memory Wall: BW Compute >> Memory >> IO/Fabric



UnifabriX

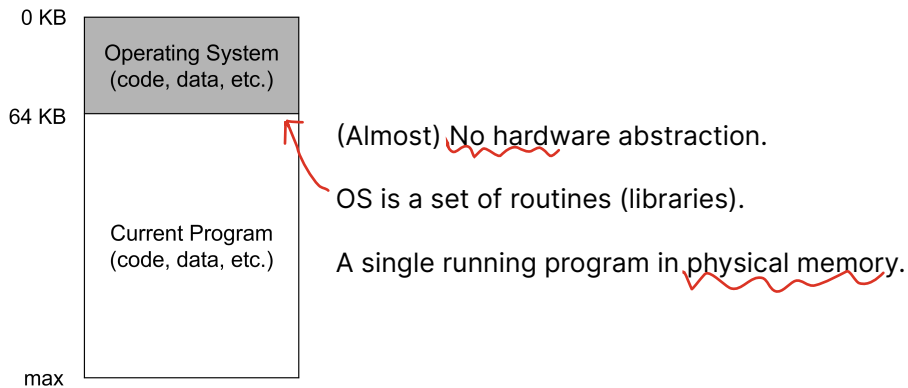
© 2024 UnifabriX. All rights reserved.

13

Memory is one of the most important topics in OS.

Hardware and software development are both very active (commercially and academically).

The early operating system

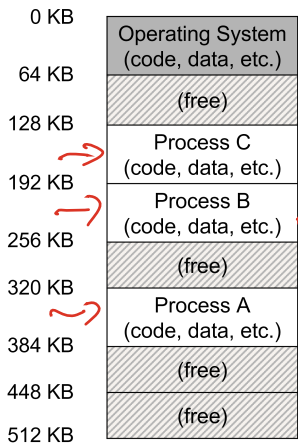


Multiple people start sharing a computer

→ Timeshare: One user runs for a bit, saves everything to disk, then load another.

But it is very slow.

Writing from disk to memory is slow...but registers are fast

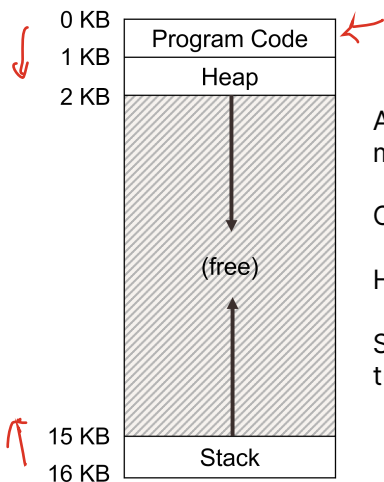


A second try - leaves everything in memory, but switch between them.

Each process gets 64 KB space from physical memory.

But there is no abstraction - any problems?

Create an abstraction – address space (VM).



An address space is the running program's view of memory in the system.

Code: where instructions live

Heap: dynamic data (malloc'd data)

Stack: local variables, arguments to routines, return values, etc.

Create an abstraction – address space

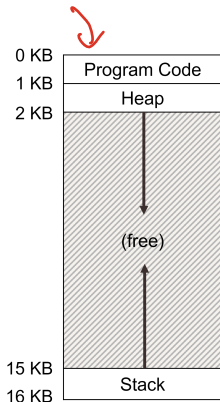
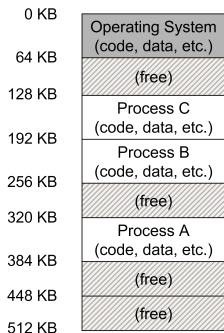
```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
    printf("location of code : %p\n", main); ←
    printf("location of heap : %p\n", malloc(100e3)); ←
    int x = 3;
    printf("location of stack: %p\n", &x); ←
    return x;
}
```

Virtual address or physical memory?

convenient. security.

Create an abstraction – address space

Difference?



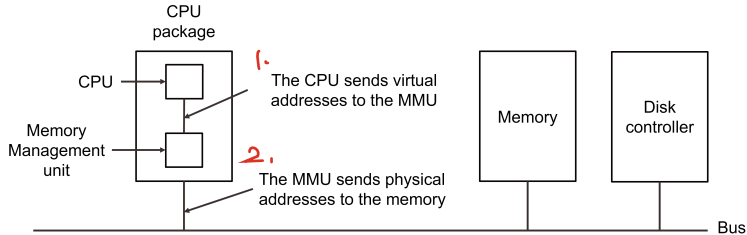
VM
→ unlimited.
private

OS.
PIN
→ 1.
→ shared
→ limited.

How can the OS build this abstraction of a private, potentially large address space for multiple running processes (all sharing memory) on top of a single, physical memory?

HW support.

The memory management unit (MMU)



How Should We Implement Virtual Mapping?

What are your ideas for mapping a process's virtual memory to physical memory?

Virtual Memory Checklist

Design. req.

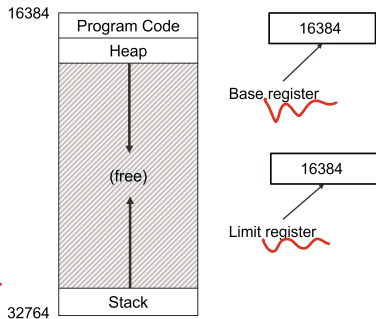
→ scalability

security

- Multiple processes must be able to co-exist
- Processes are not aware they are sharing physical memory
- Processes cannot access each others data (unless allowed explicitly)
- Performance close to using physical memory ← performance (speed)
- Limit the amount of fragmentation (wasted memory)

→ efficiency of usage.

Base and limit registers



The simple solution: Each CPU has two special hardware registers: base and limit.

When a process is run, the addresses are loaded to these registers.

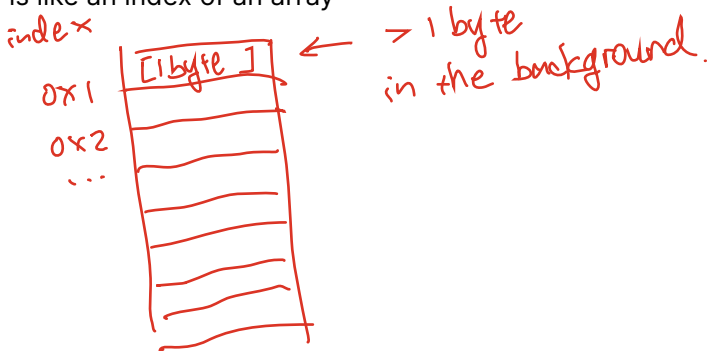
An addition and a comparison on every memory reference.

Remember That Memory is Byte Addressable

The smallest unit you can use to address memory is one byte

You can read or write one byte at a time at minimum

Each "address" is like an index of an array



Segmentation or Segments are Coarse Grained

Divide the virtual address space into segments for: code, data, stack, and heap H(W)

Each segment is a variable size, and can be dynamically resized

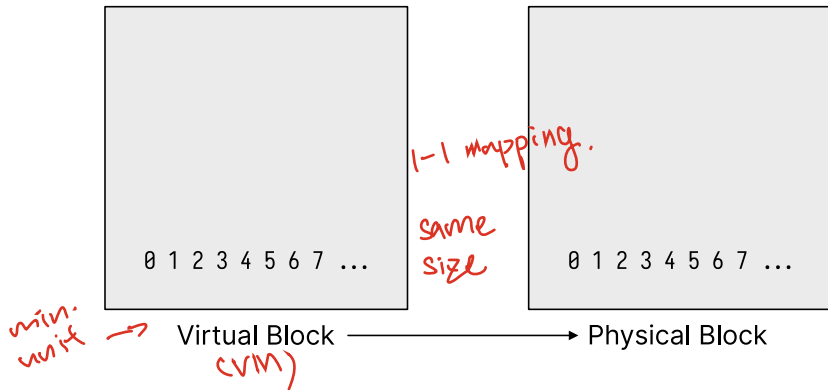
This is an old legacy technique that's no longer used

Segments can be large and very costly to relocate ←

It also leads to fragmentation (gaps of unused memory)

No longer used in modern operating systems

First Insight: Divide Memory into Fixed-Sized Chunks



Paging

One technique is to divide memory up into fixed-size pages (typically 4096 bytes or 4 KB)

- A page in virtual memory is called a page ↲
- A page in physical memory is called a frame ↲

A Library Analogy

Imagine you are in a rare book library

- You can't browse the shelves yourself ←
- You can only read books in the reading room. ←

} It process's view
of memory

→ You have to make a request for the records to be transferred to the reading room.

- You have to request at least a page at a time.

multiples of 1 page

The librarian puts the request through, the system fetches it from the back room.
↓ OS. MMU.

Once the book is in the reading room, you can read a word (byte) at a time.
memory.

You Typically Do Not Use All 64 Virtual Address Bits

CPUs may have different levels of virtual addresses you can use
Implementation ideas are the same

39-bit

48-bit

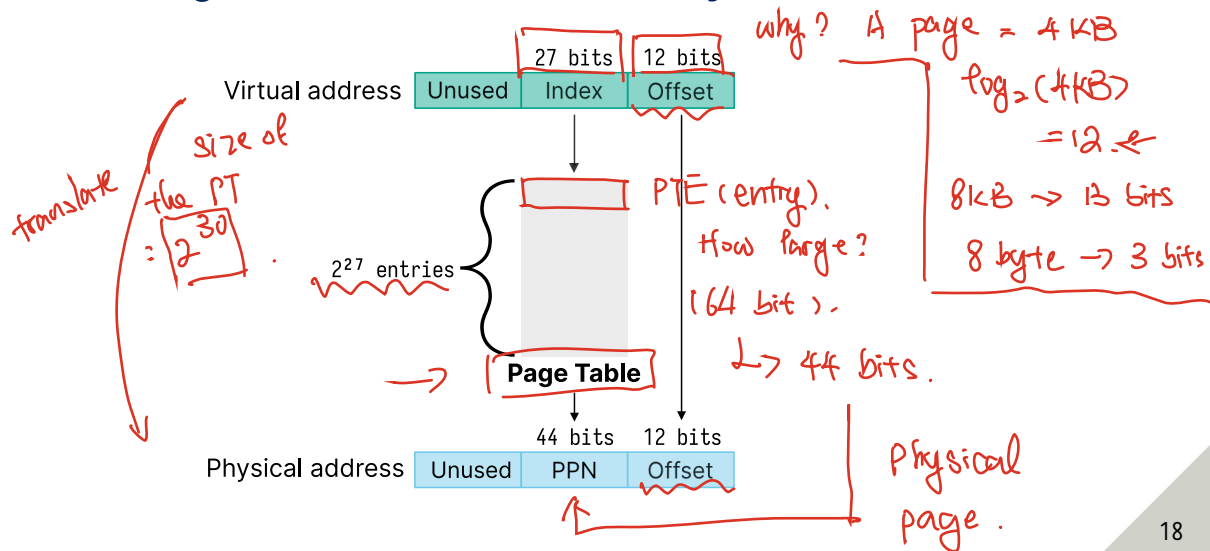
We'll assume a 39 bit virtual address space used by RISC-V and other architectures

Allows for 512 GiB of addressable memory (called Sv39)

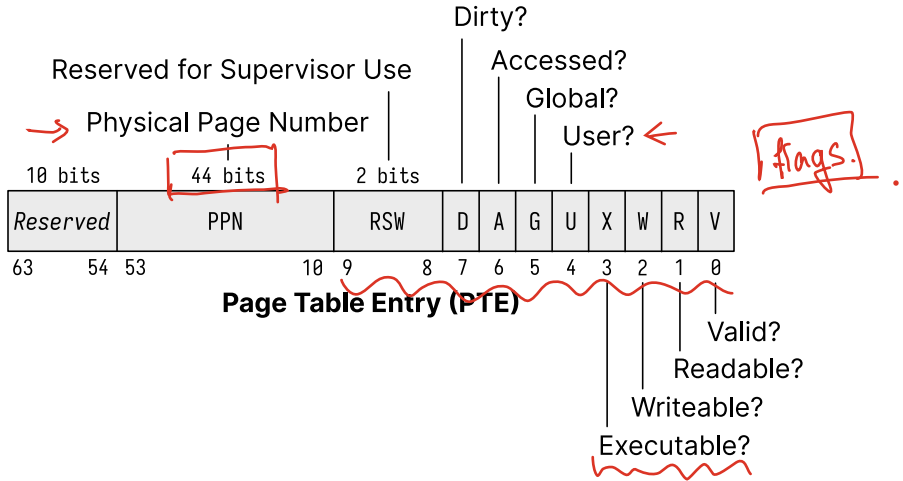
- Implemented with a page table indexed by Virtual Page Number (VPN)
Looks up the Physical Page Number (PPN)

$$27 + 12 = 39 \text{ (SV 39)}.$$

The Page Table Translates Virtual to Physical Addresses



The Page Table Entry (PTE) Also Stores Flags in the Lower Bits



The Kernel Handles Translating Virtual Addresses

Considering the following page table:

4 entries

VPN	PPN
0x0	0x1
0x1	0x4
0x2	0x3
0x3	0x7

4KB page
↳ 12 bits offset
Hex number. 0x0 0b0000
0xF 0b1111
3 hex numbers.

We would get the following virtual → physical address translations:

0x0AB0	→	0x1AB0.	1. offset → same
0x1FA0	→		2. VPN → PPN
0x2884	→	0x3884.	
0x32D0	→		

Page Translation Example Problem

step. 1. page size 64 bytes -

→ 6 bits of offset.

step. 2: $8 - 6 = 2$ VP# bits -

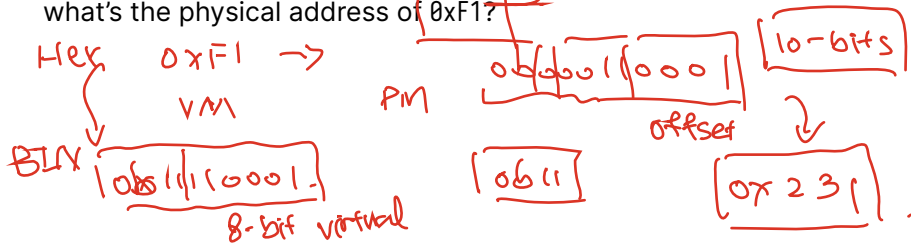
Assume you have a 8-bit virtual address, 10-bit physical address and each page is 64 bytes

- How many virtual pages are there? $2^2 = 4$
- How many physical pages are there? $2^4 = 16$
- How many entries are in the page table? 4
- Given the page table is $[0x2, 0x5, 0x1, 0x8]$ what's the physical address of $0xF1$?

Physical page -

6-bit offset

$10 - 6 = 4$ bits PPN



Page Translation Example Problem

Assume you have a 8-bit virtual address, 10-bit physical address and each page is 64 bytes

- How many virtual pages are there? $\frac{2^8}{2^6} = 4$
- How many physical pages are there? $\frac{2^{10}}{2^6} = 16$
- How many entries are in the page table? 4
- Given the page table is [0x2, 0x5, 0x1, 0x8]
what's the physical address of 0xF1?
0x231

Each Process Gets Its Own Page Table

When you fork a process, it will copy the page table from the parent

Turn off the write permission so the kernel can implement copy-on-write

The problem is there are 2^{27} entries in the page table, each one is 8 bytes

This means the page table would be 1 GB ←

Note that RISC-V translates a 39-bit virtual to a 56-bit physical address

It has 10 bits to spare in the PTE and could expand

Page size is 4096 bytes (size of offset field)

64-bit

We Use Pages for Memory Translation

Divide memory into blocks, so we only have to translate once per block

Use page tables (array of PTEs) to access the PPN (and flags)

New problem: these page tables are always huge!